

Q8: Surface Defect Detection using YOLOv8

Objective

Build an object detection model to automatically detect defects on steel/metal surfaces using the NEU Surface Defect Dataset.

Dataset: <https://universe.roboflow.com/pavithraa-sekar/neu-surface-defect-dataset>

- ✓ Part A: Data Preparation (25 Marks)
- ✓ Step 1: Install Required Libraries

```
!pip install ultralytics  
!pip install roboflow
```

```

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.12/dist
Requirement already satisfied: pi-heif<2 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: pillow-avif-plugin<2 in /usr/local/lib/python3.
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/di
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.12/dist
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.12/di
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: requests_toolbelt in /usr/local/lib/python3.12/
Requirement already satisfied: filetype in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: typer<0.26,>=0.12.0 in /usr/local/lib/python3.1
Requirement already satisfied: click>=8.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyth
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/pytho
Requirement already satisfied: mdurl<=0.1 in /usr/local/lib/python3.12/dist-pa

```

✓ Step 2: Download Dataset from Roboflow

```

from roboflow import Roboflow

rf = Roboflow(api_key="xFPQpxd6Ubz3SjZaACUy")

project = rf.workspace("pavithraa-sekar").project(
    "neu-surface-defect-dataset"
)

dataset = project.version(1).download("yolov8")

```

```

loading Roboflow workspace...
loading Roboflow project...

```

✓ Verify Dataset Structure

```
import os
```

```
dataset_path = dataset.location  
print(os.listdir(dataset_path))
```

```
['README.dataset.txt', 'data.yaml', 'README.roboflow.txt', 'valid', 'train']
```

Expected output:

```
train/  
valid/  
test/  
data.yaml
```

✓ Verify Images and Labels

```
train_images = os.listdir(f"{dataset_path}/train/images")  
train_labels = os.listdir(f"{dataset_path}/train/labels")  
  
print("Images :", len(train_images))  
print("Labels :", len(train_labels))
```

```
Images : 1439  
Labels : 1439
```

✓ Verify data.yaml

```
with open(f"{dataset_path}/data.yaml") as f:  
    print(f.read())
```

```
names:  
- crazing  
- inclusion  
- patches  
- pitted_surface  
- rolled-in_scale  
- scratches  
nc: 6  
roboflow:  
  license: CC BY 4.0  
  project: neu-surface-defect-dataset  
  url: https://universe.roboflow.com/pavithraa-sekar/neu-surface-defect-dataset/  
  version: 1  
  workspace: pavithraa-sekar  
test: ../test/images  
train: ../train/images  
val: ../valid/images
```

✓ Display Sample Images with Bounding Boxes

```
import cv2
import matplotlib.pyplot as plt
import random
import os

image_dir = f"{dataset_path}/train/images"
label_dir = f"{dataset_path}/train/labels"

samples = random.sample(os.listdir(image_dir), 3)

for file in samples:
    img_path = os.path.join(image_dir, file)
    label_path = os.path.join(label_dir, file.replace(".jpg", ".txt"))

    image = cv2.imread(img_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    h,w,_ = image.shape

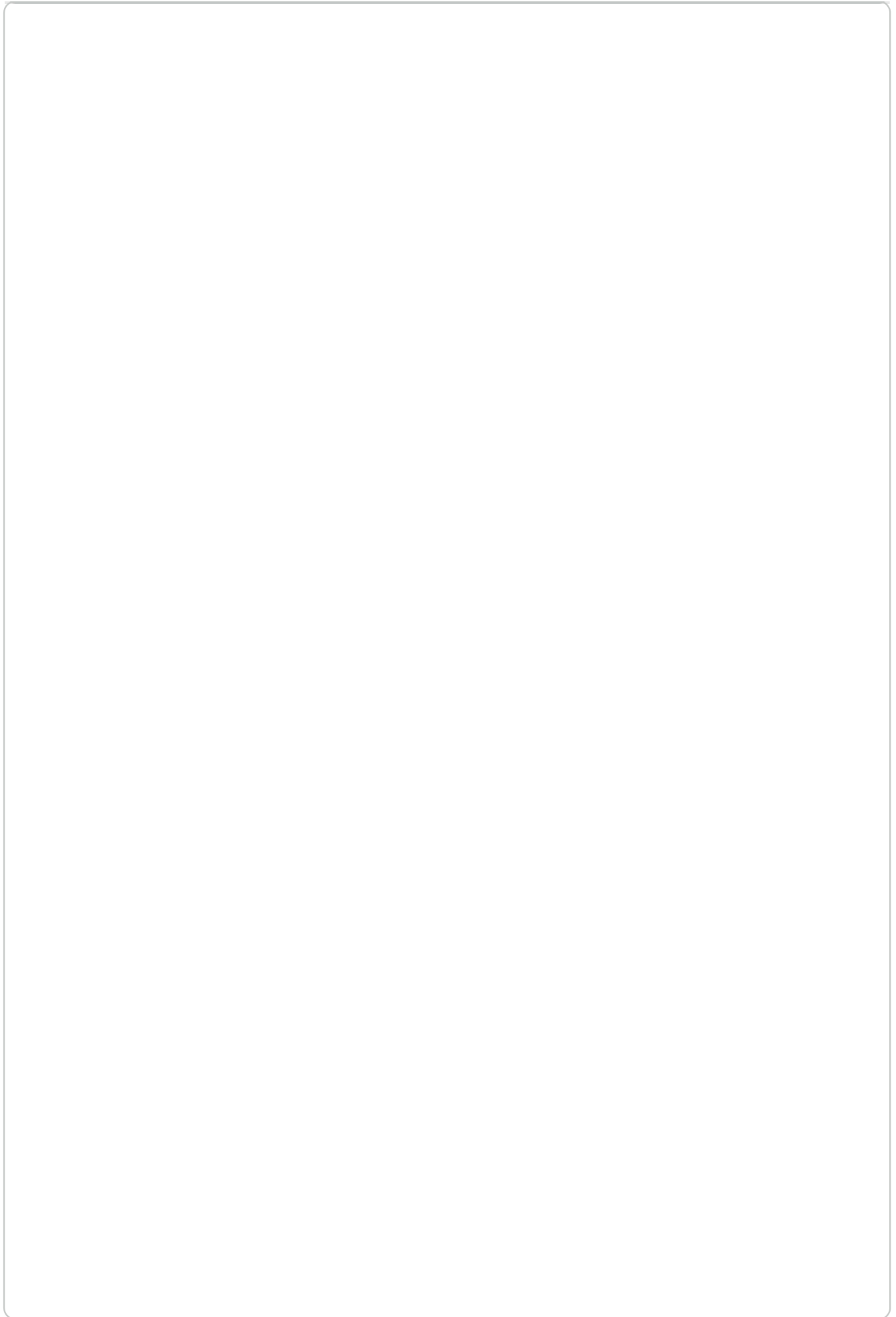
    with open(label_path) as f:
        boxes = f.readlines()

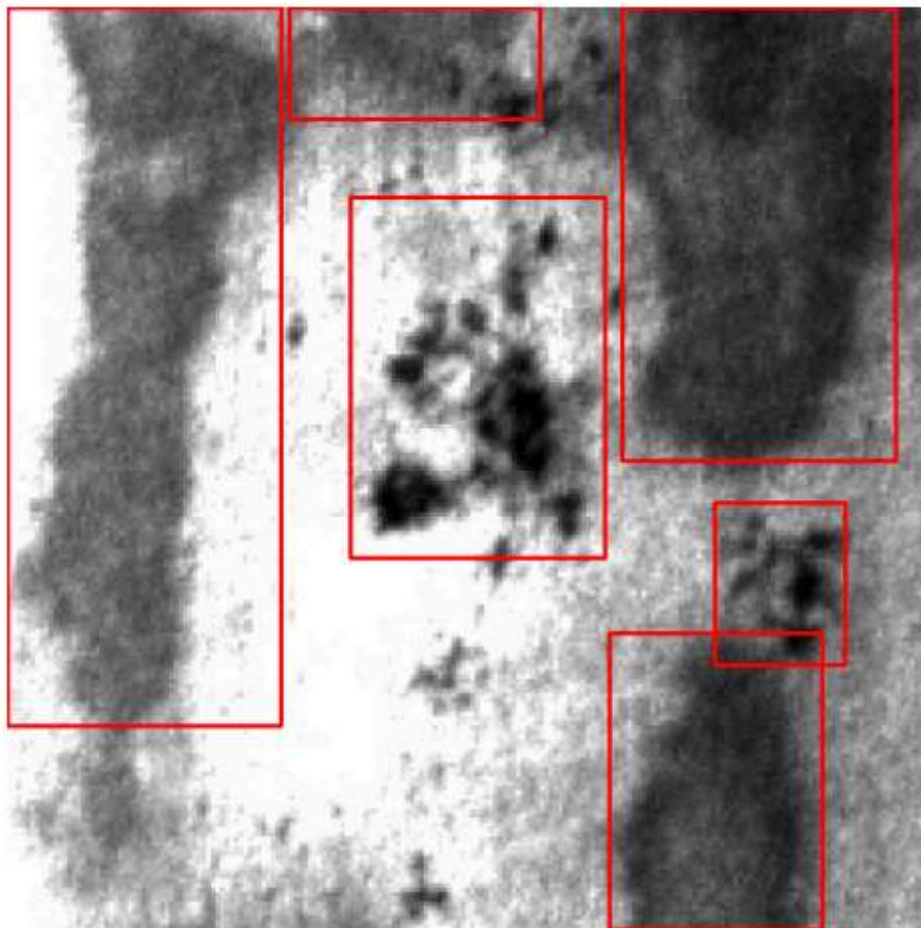
    for box in boxes:
        cls,x,y,bw,bh = map(float, box.split())

        x1 = int((x-bw/2)*w)
        y1 = int((y-bh/2)*h)
        x2 = int((x+bw/2)*w)
        y2 = int((y+bh/2)*h)

        cv2.rectangle(image,(x1,y1),(x2,y2),(255,0,0),2)

plt.figure(figsize=(6,6))
plt.imshow(image)
plt.axis("off")
plt.show()
```





- Part B: Model Training and Evaluation (10 Marks)

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n.pt")
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt>

```
results = model.train(
    data=f"{dataset_path}/data.yaml",
    epochs=20,
    imgsz=640,
    batch=16
)
```

```

UltraLytics 8.4.80 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 1491
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_a
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultraly
Overriding model.yaml nc=80 with nc=6

```

	from	n	params	module
0	-1	1	464	ultralytics.nn.modules.conv.Conv
1	-1	1	4672	ultralytics.nn.modules.conv.Conv

```
results = model.val()
print(results)
```



```

0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.0
0.072072, 0.073073, 0.074074, 0.075075, 0.076076, 0.0
0.096096, 0.097097, 0.098098, 0.099099, 0.1001, 0
0.12012, 0.12112, 0.12212, 0.12312, 0.12412, 0.
0.14414, 0.14515, 0.14615, 0.14715, 0.14815, 0.
0.16817, 0.16917, 0.17017, 0.17117, 0.17217, 0.
0.19219, 0.19319, 0.19419, 0.1952, 0.1962, 0
0.21622, 0.21722, 0.21822, 0.21922, 0.22022, 0.
0.24024, 0.24124, 0.24224, 0.24324, 0.24424, 0.
0.26426, 0.26527, 0.26627, 0.26727, 0.26827, 0.
0.28829, 0.28929, 0.29029, 0.29129, 0.29229, 0.
0.31231, 0.31331, 0.31431, 0.31532, 0.31632, 0.
0.33634, 0.33734, 0.33834, 0.33934, 0.34034, 0.
0.36036, 0.36136, 0.36236, 0.36336, 0.36436, 0.
0.38438, 0.38539, 0.38639, 0.38739, 0.38839, 0.
0.40841, 0.40941, 0.41041, 0.41141, 0.41241, 0.
0.43243, 0.43343, 0.43443, 0.43544, 0.43644, 0.
0.45646, 0.45746, 0.45846, 0.45946, 0.46046, 0.
0.48048, 0.48148, 0.48248, 0.48348, 0.48448, 0.
0.5045, 0.50551, 0.50651, 0.50751, 0.50851, 0.
0.52853, 0.52953, 0.53053, 0.53153, 0.53253, 0.
0.55255, 0.55355, 0.55455, 0.55556, 0.55656, 0.
0.57658, 0.57758, 0.57858, 0.57958, 0.58058, 0.
0.6006, 0.6016, 0.6026, 0.6036, 0.6046, 0.
0.62462, 0.62563, 0.62663, 0.62763, 0.62863, 0.
0.64865, 0.64965, 0.65065, 0.65165, 0.65265, 0.
0.67267, 0.67367, 0.67467, 0.67568, 0.67668, 0.
0.6967, 0.6977, 0.6987, 0.6997, 0.7007, 0
0.72072, 0.72172, 0.72272, 0.72372, 0.72472, 0.
0.74474, 0.74575, 0.74675, 0.74775, 0.74875, 0.
0.76877, 0.76977, 0.77077, 0.77177, 0.77277, 0.
0.79279, 0.79379, 0.79479, 0.7958, 0.7968, 0
0.81682, 0.81782, 0.81882, 0.81982, 0.82082, 0.
0.84084, 0.84184, 0.84284, 0.84384, 0.84484, 0.
0.86486, 0.86587, 0.86687, 0.86787, 0.86887, 0.
0.88889, 0.88989, 0.89089, 0.89189, 0.89289, 0.
0.91291, 0.91391, 0.91491, 0.91592, 0.91692, 0.
0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.
0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.
0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.
[ 0.98113, 0.98113, 0.97484, ..., 0, 0,
[ 1, 1, 0.99482, ..., 0, 0,
[ 0.98851, 0.98851, 0.98851, ..., 0, 0,
[ 0.99242, 0.99242, 0.99242, ..., 0, 0,
[ 0.99174, 0.99174, 0.99174, ..., 0, 0,
fitness: 0.3727074751096771
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metri
maps: array([ 0.14652, 0.44304, 0.58458, 0.42301, 0.23067,
names: {0: 'crazing', 1: 'inclusion', 2: 'patches', 3: 'pitted_surface', 4: 'r
nt_per_class: array([162, 159, 193, 87, 132, 121])
nt_per_image: array([60, 71, 73, 60, 60, 60])
results_dict: {'metrics/precision(B)': 0.6722415195146662, 'metrics/recall(B)'
save_dir: PosixPath('/content/runs/detect/val')
speed: {'preprocess': 2.2668637388891155, 'inference': 4.0837926277763215, 'lo
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img':

```


✓ Important Metrics

- Precision
- Recall
- [mAP@0.5](#)
- [mAP@0.5:0.95](#)

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Measures how many predicted defects are actually defects.

Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Measures how many actual defects are detected.

[mAP@0.5](#)

Mean Average Precision at IoU threshold of 0.5.

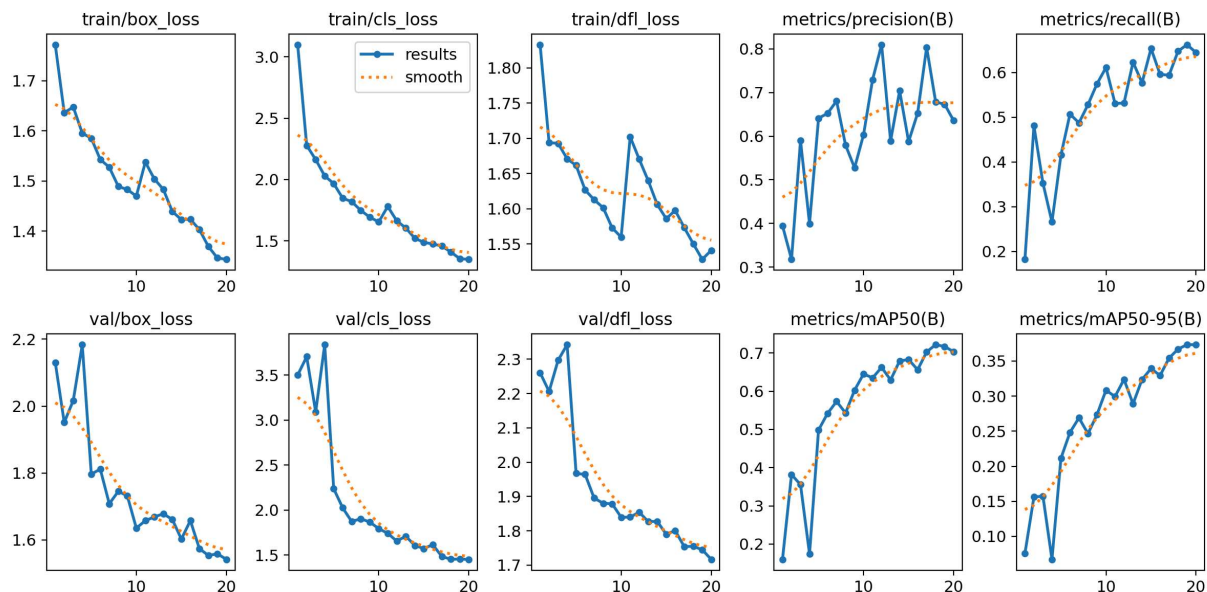
[mAP@0.5:0.95](#)

Average mAP computed from IoU thresholds 0.5 to 0.95.

✓ Display Training Curves

```
from IPython.display import Image

Image(filename="runs/detect/train/results.png")
```



✓ Part C: Model Inference

```
best_model = YOLO(
    "runs/detect/train/weights/best.pt"
)
```

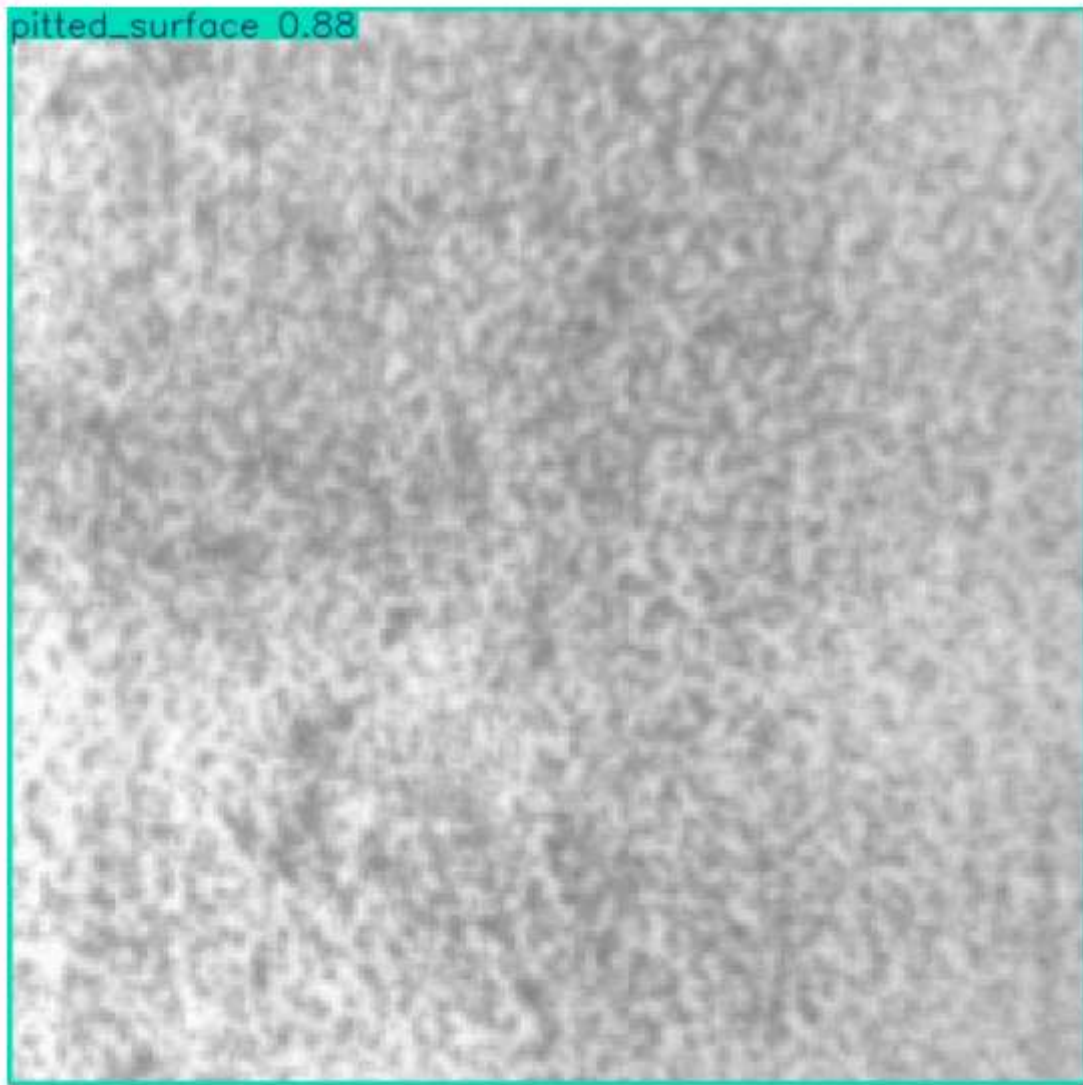
```
results = best_model.predict(
    source=f"{dataset_path}/valid/images", # Changed from test/images to valid/
    save=True,
    conf=0.25
)
```

```
image 319/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_259.
image 320/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_260.
image 321/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_261.
image 322/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_262.
image 323/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_263.
image 324/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_264.
image 325/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_265.
image 326/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_266.
image 327/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_267.
image 328/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_268.
image 329/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_269.
image 330/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_270.
image 331/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_271.
image 332/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_272.
image 333/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_273.
image 334/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_274.
image 335/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_275.
image 336/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_276.
image 337/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_277.
image 338/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_278.
image 339/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_279.
image 340/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_280.
image 341/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_281.
image 342/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_282.
image 343/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_283.
image 344/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_284.
image 345/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_285.
image 346/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_286.
image 347/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_287.
image 348/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_288.
image 349/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_289.
image 350/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_290.
image 351/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_291.
image 352/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_292.
image 353/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_293.
image 354/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_294.
image 355/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_295.
image 356/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_296.
image 357/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_297.
image 358/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_298.
image 359/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_299.
image 360/360 /content/NEU-Surface-Defect-Dataset-1/valid/images/scratches_300.
Speed: 2.3ms preprocess, 7.4ms inference, 1.3ms postprocess per image at shape
Results saved to /content/runs/detect/predict
```

```
from glob import glob
import matplotlib.pyplot as plt

pred_images = glob("runs/detect/predict/*.jpg")

for img in pred_images[:3]:
    plt.figure(figsize=(7,7))
    plt.imshow(plt.imread(img))
    plt.axis("off")
    plt.show()
```

```
import shutil

for i,img in enumerate(pred_images[:3]):
    shutil.copy(img, f"Q8_output_{i+1}.jpg")
```

```
best_model.predict(
    source="patches_205.jpg",
    save=True,
    conf=0.25
)
```

